AI Assistant Coding Assignment-16.3

NAME:K.Trinay Prasad

BATCH:32

HNO:2303A52070

Task 1 – Hospital Management Database

Task:

Create schema and queries for a Hospital Management System.

Instructions:

• Tables: Doctors, Patients, Appointments.

• Use AI to define constraints (unique IDs, valid dates).

• Generate queries:

• List all appointments for a specific doctor.

• Retrieve patient history by patient ID.

• Count total patients treated by each doctor.

Expected Output:

• Normalized schema and SQL queries with joins.

```sql
--Create tables for doctors , patients and appointments
CREATE TABLE Doctors (
    doctor_id INT PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    specialty VARCHAR(100) NOT NULL
);
CREATE TABLE Patients (
    patient_id INT PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    age INT NOT NULL
);
CREATE TABLE Appointments (
    appointment_id INT PRIMARY KEY,
    doctor_id INT,
    patient_id INT,
    appointment_date DATE NOT NULL,
    FOREIGN KEY (doctor_id) REFERENCES Doctors(doctor_id),
    FOREIGN KEY (patient_id) REFERENCES Patients(patient_id)
);
--Insert data into doctors tables
INSERT INTO Doctors (doctor_id, name, specialty) VALUES (1, 'Dr. Smith',
'Cardiology');
INSERT INTO Doctors (doctor_id, name, specialty) VALUES (2, 'Dr. Johnson',
'Neurology');
```

```sql
INSERT INTO Doctors (doctor_id, name, specialty) VALUES (3, 'Dr. Brown',
'Pediatrics');
--Insert data into patients table
INSERT INTO Patients (patient_id, name, age) VALUES (1, 'Alice', 30);
INSERT INTO Patients (patient_id, name, age) VALUES (2, 'Bob', 25);
INSERT INTO Patients (patient_id, name, age) VALUES (3, 'Charlie', 35);
--Insert data into appointments table
INSERT INTO Appointments (appointment_id, doctor_id, patient_id,
appointment_date) VALUES (1, 1, 1, '2024-01-15');
INSERT INTO Appointments (appointment_id, doctor_id, patient_id,
appointment_date) VALUES (2, 1, 2, '2024-01-20');
INSERT INTO Appointments (appointment_id, doctor_id, patient_id,
appointment_date) VALUES (3, 2, 1, '2024-02-10');
---List all appointments for a specific doctor.
SELECT a.appointment_id, d.name AS doctor_name, p.name AS patient_name,
a.appointment_date
FROM Appointments a
JOIN Doctors d ON a.doctor_id = d.doctor_id
JOIN Patients p ON a.patient_id = p.patient_id
WHERE d.name = 'Dr. Smith';
--Retrieve patient history by patient ID.
SELECT a.appointment_id, d.name AS doctor_name, a.appointment_date
FROM Appointments a
JOIN Doctors d ON a.doctor_id = d.doctor_id
WHERE a.patient_id = 1;
--Count total patients treated by each doctor.
SELECT d.name AS doctor_name, COUNT(a.patient_id) AS total_patients
FROM Doctors d LEFT JOIN Appointments a ON d.doctor_id = a.doctor_id
GROUP BY d.name;
```

SQL ▼ 1 / 1 1 - 2 of 2

appointment_id	doctor_name	patient_name	appointment_
1	Dr. Smith	Alice	2024-01-15
2	Dr. Smith	Bob	2024-01-20

SQL ▼ 1 / 1 1 - 2 of 2

appointment_id	doctor_name	appointment_date
1	Dr. Smith	2024-01-15
3	Dr. Johnson	2024-02-10

SQL ▼ 1 / 1 1 - 3 of 3

doctor_name	total_patients
Dr. Brown	0
Dr. Johnson	1
Dr. Smith	2

Task 2 – Real-Time Application: Online Shopping Database

Scenario:

Design a database for an E-commerce platform.

Requirements:

• Tables: Users, Products, Orders, OrderDetails.

• Generate queries to:

• Retrieve all orders by a user.

• Find the most purchased product.

• Calculate total revenue in a given month.

• AI should also suggest normalization improvements and query optimization.

Expected Output:

• Complete schema with relationships + SQL queries for analytics.

```sql
-- Create tables for users , products , orders and orderdetails
CREATE TABLE users (
    user_id INT PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    email VARCHAR(100) NOT NULL
);
CREATE TABLE products (
    product_id INT PRIMARY KEY,
    product_name VARCHAR(100) NOT NULL,
    price DECIMAL(10, 2) NOT NULL
);
CREATE TABLE orders (
    order_id INT PRIMARY KEY,
    user_id INT,
    order_date DATE NOT NULL,
    FOREIGN KEY (user_id) REFERENCES users(user_id)
);
CREATE TABLE orderdetails (
    orderdetail_id INT PRIMARY KEY,
    order_id INT,
    product_id INT,
    quantity INT NOT NULL,
    FOREIGN KEY (order_id) REFERENCES orders(order_id),
    FOREIGN KEY (product_id) REFERENCES products(product_id)
);
-- Insert data into users table
INSERT INTO users (user_id, name, email) VALUES (1, 'Alice',
'alice@example.com');
INSERT INTO users (user_id, name, email) VALUES (2, 'Bob', 'bob@example.com');
-- Insert data into products table
INSERT INTO products (product_id, product_name, price) VALUES (1, 'Laptop',
1000.00);
INSERT INTO products (product_id, product_name, price) VALUES (2, 'Mouse',
25.00);
INSERT INTO products (product_id, product_name, price) VALUES (3, 'Keyboard',
75.00);
-- Insert data into orders table
INSERT INTO orders (order_id, user_id, order_date) VALUES (1, 1, '2024-01-
15');
```

```sql
INSERT INTO orders (order_id, user_id, order_date) VALUES (2, 2, '2024-01-
20');
-- Insert data into orderdetails table
INSERT INTO orderdetails (orderdetail_id, order_id, product_id, quantity)
VALUES (1, 1, 1, 1);
INSERT INTO orderdetails (orderdetail_id, order_id, product_id, quantity)
VALUES (2, 1, 2, 2);
INSERT INTO orderdetails (orderdetail_id, order_id, product_id, quantity)
VALUES (3, 2, 1, 1);
INSERT INTO orderdetails (orderdetail_id, order_id, product_id, quantity)
VALUES (4, 2, 3, 1);

-- Retrieve all the orders by a specific user.
SELECT o.order_id, o.order_date, p.product_name, od.quantity
FROM orders o JOIN orderdetails od ON o.order_id = od.order_id
JOIN products p ON od.product_id = p.product_id
WHERE o.user_id = 1;

-- Display the most purschased product.
SELECT p.product_name, SUM(od.quantity) AS total_quantity
FROM products p JOIN orderdetails od ON p.product_id = od.product_id
GROUP BY p.product_name
ORDER BY total_quantity DESC
LIMIT 1;

--calculate the total revenue generated in a specific month.
SELECT SUM(p.price * od.quantity) AS total_revenue
FROM orders o JOIN orderdetails od ON o.order_id = od.order_id
JOIN products p ON od.product_id = p.product_id
WHERE MONTH(o.order_date) = 1 AND YEAR(o.order_date) = 2024;
```

SQL ▼ 1 / 1 1 - 2 of 2

order_id	order_date	product_name	quantity
1	2024-01-15	Laptop	1
1	2024-01-15	Mouse	2

SQL ▼ 1 / 1 1 - 1 of 1

product_name	total_quantity
Laptop	2

Task 3 – Library Database

Task:

Design schema for a Library Management System.

Instructions:

• Tables: Books, Members, Loans.

• Use AI to suggest indexing strategy for faster queries.

• Generate queries:

• Retrieve all books currently issued.

• Find overdue books (loan date > 30 days).

• Count number of books loaned by each member.

Expected Output:

• Schema + SQL queries demonstrating joins and conditions.

```
--Create tables for books , members and loans
CREATE TABLE books (
    book_id INT PRIMARY KEY,
    title VARCHAR(100) NOT NULL,
    author VARCHAR(100) NOT NULL
);
CREATE TABLE members (
    member_id INT PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    email VARCHAR(100) NOT NULL
);
CREATE TABLE loans (
    loan_id INT PRIMARY KEY,
    book_id INT,
    member_id INT,
    loan_date DATE NOT NULL,
    return_date DATE,
    FOREIGN KEY (book_id) REFERENCES books(book_id),
    FOREIGN KEY (member_id) REFERENCES members(member_id)
);
--Insert data into books table
INSERT INTO books (book_id, title, author) VALUES (1, 'The Great Gatsby', 'F.
Scott Fitzgerald');
INSERT INTO books (book_id, title, author) VALUES (2, 'To Kill a Mockingbird',
'Harper Lee');
INSERT INTO books (book_id, title, author) VALUES (3, '1984', 'George
Orwell');
```

```sql
--Insert data into members table
INSERT INTO members (member_id, name, email) VALUES (1, 'Alice',
'alice@example.com');
INSERT INTO members (member_id, name, email) VALUES (2, 'Bob',
'bob@example.com');
INSERT INTO members (member_id, name, email) VALUES (3, 'Charlie',
'charlie@example.com');
--Insert data into loans table
INSERT INTO loans (loan_id, book_id, member_id, loan_date, return_date) VALUES
(1, 1, 1, '2024-01-15', '2024-01-30');
INSERT INTO loans (loan_id, book_id, member_id, loan_date, return_date) VALUES
(2, 2, 1, '2024-02-01', NULL);
INSERT INTO loans (loan_id, book_id, member_id, loan_date, return_date) VALUES
(3, 3, 2, '2024-01-20', '2024-02-05');

--List all the books currently issued to a specific member.
SELECT b.title, b.author, l.loan_date
FROM loans l JOIN books b ON l.book_id = b.book_id
WHERE l.member_id = 1 AND l.return_date IS NULL;

--find overdue books where loan date ismore than 30 days
SELECT b.title, b.author, l.loan_date
FROM loans l JOIN books b ON l.book_id = b.book_id
WHERE l.return_date IS NULL AND DATEDIFF(CURDATE(), l.loan_date) > 30;

--count number of booked loanes by each member
SELECT m.name AS member_name, COUNT(l.loan_id) AS total_loans
FROM members m LEFT JOIN loans l ON m.member_id = l.member_id
GROUP BY m.name;
```

SQL ▼ < 1 / 1 > 1 - 0 of 0

SQL ▼ < 1 / 1 > 1 - 0 of 0

SQL ▼ < 1 / 1 > 1 - 0 of 0

SQL ▼ < 1 / 1 > 1 - 0 of 0

SQL ▼ < 1 / 1 > 1 - 0 of 0

SQL ▼ < 1 / 1 > 1 - 0 of 0

SQL ▼ < 1 / 1 > 1 - 0 of 0

SQL ▼ < 1 / 1 > 1 - 0 of 0

SQL ▼ < 1 / 1 > 1 - 0 of 0

SQL ▼ < 1 / 1 > 1 - 0 of 0

SQL ▼ < 1 / 1 > 1 - 0 of 0

SQL ▼ < 1 / 1 > 1 - 1 of 1

title	author	loan_date
To Kill a Mockingbird	Harper Lee	2024-02-01

Task 4: Optimize Queries

Instructions:

• Use AI tools to analyze query efficiency.

• Optimize inefficient queries using joins, indexes, or reducing subqueries.

• Test optimized queries for correctness.

Starter Code Example:

-- Original query

SELECT * FROM Books WHERE author_id IN (SELECT author_id FROM

Authors WHERE country='UK');

-- Optimized query

SELECT b.*

FROM Books b

JOIN Authors a ON b.author_id = a.author_id

WHERE a.country = 'UK';

Expected Output:

• Both queries return the same result: all books written by UK authors.

• Optimized query performs faster for larger datasets.

```
/*SELECT *
FROM Books
WHERE author_id IN (
    SELECT author_id
    FROM Authors
    WHERE country = 'UK'
);/*/
--modify the above query to use JOIN instead of subquery

SELECT b.*
FROM Books b
JOIN Authors a ON b.author_id = a.author_id
WHERE a.country = 'UK';
```

Task 5: University Course Registration

Scenario:

SR University needs a course registration system for students enrolling in different courses under faculty supervision.

• Use Copilot to design schema tables: Students, Courses, Faculty, and Registrations.

• Define relationships:

• Each student can register for multiple courses.

• Each course is taught by a faculty member.

• Write SQL queries to:

• List all students enrolled in a specific course.

• Find faculty members teaching more than 2 courses.

• Retrieve courses with the highest number of registrations.

```sql
--Create tables for Students, Courses, Faculty, and Registrations.
CREATE TABLE students (
    student_id INT PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    email VARCHAR(100) NOT NULL
);
CREATE TABLE courses (
    course_id INT PRIMARY KEY,
    course_name VARCHAR(100) NOT NULL,
    faculty_id INT,
    FOREIGN KEY (faculty_id) REFERENCES faculty(faculty_id)
);
CREATE TABLE faculty (
    faculty_id INT PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    department VARCHAR(100) NOT NULL
);
CREATE TABLE registrations (
    registration_id INT PRIMARY KEY,
    student_id INT,
    course_id INT,
    registration_date DATE NOT NULL,
    FOREIGN KEY (student_id) REFERENCES students(student_id),
    FOREIGN KEY (course_id) REFERENCES courses(course_id)
);
--Insert data into students table
INSERT INTO students (student_id, name, email) VALUES (1, 'Alice',
'alice@example.com');
```

```sql
INSERT INTO students (student_id, name, email) VALUES (2, 'Bob',
'bob@example.com');
INSERT INTO students (student_id, name, email) VALUES (3, 'Charlie',
'charlie@example.com');
--Insert data into faculty table
INSERT INTO faculty (faculty_id, name, department) VALUES (1, 'Dr. Smith',
'Computer Science');
INSERT INTO faculty (faculty_id, name, department) VALUES (2, 'Dr. Johnson',
'Mathematics');
INSERT INTO faculty (faculty_id, name, department) VALUES (3, 'Dr. Brown',
'Physics');
--Insert data into courses table
INSERT INTO courses (course_id, course_name, faculty_id) VALUES (1, 'Database
Systems', 1);
INSERT INTO courses (course_id, course_name, faculty_id) VALUES (2,
'Calculus', 2);
INSERT INTO courses (course_id, course_name, faculty_id) VALUES (3, 'Quantum
Mechanics', 3);
--Insert data into registrations table
INSERT INTO registrations (registration_id, student_id, course_id,
registration_date) VALUES (1, 1, 1, '2024-01-15');
INSERT INTO registrations (registration_id, student_id, course_id,
registration_date) VALUES (2, 1, 2, '2024-01-20');
INSERT INTO registrations (registration_id, student_id, course_id,
registration_date) VALUES (3, 2, 1, '2024-01-25');
INSERT INTO registrations (registration_id, student_id, course_id,
registration_date) VALUES (4, 3, 3, '2024-01-30');

--List all students enrolled in a specific course.
SELECT s.name AS student_name, c.course_name
FROM registrations r JOIN students s ON r.student_id = s.student_id
JOIN courses c ON r.course_id = c.course_id
WHERE c.course_name = 'Database Systems';

--Find faculty members teaching more than 2 courses.
SELECT f.name AS faculty_name, COUNT(c.course_id) AS total_courses
FROM faculty f JOIN courses c ON f.faculty_id = c.faculty_id
GROUP BY f.name
HAVING COUNT(c.course_id) > 2;

--Retrieve courses with the highest number of registration
SELECT c.course_name, COUNT(r.registration_id) AS total_registrations
FROM courses c JOIN registrations r ON c.course_id = r.course_id
GROUP BY c.course_name
ORDER BY total_registrations DESC
LIMIT 1;
```

SQL ▼ < 1 / 1 > 1 - 0 of 0

SQL ▼ < 1 / 1 > 1 - 0 of 0

SQL ▼ < 1 / 1 > 1 - 0 of 0

SQL ▼ < 1 / 1 > 1 - 0 of 0

SQL ▼ < 1 / 1 > 1 - 0 of 0

SQL ▼ < 1 / 1 > 1 - 0 of 0

SQL ▼ < 1 / 1 > 1 - 0 of 0

SQL ▼ < 1 / 1 > 1 - 0 of 0

SQL ▼ < 1 / 1 > 1 - 0 of 0

SQL ▼ < 1 / 1 > 1 - 0 of 0

SQL ▼ < 1 / 1 > 1 - 0 of 0

SQL ▼ < 1 / 1 > 1 - 0 of 0

SQL ▼ < 1 / 1 > 1 - 0 of 0

SQL ▼ < 1 / 1 > 1 - 0 of 0

SQL ▼ < 1 / 1 > 1 - 0 of 0

SQL ▼ < 1 / 1 > 1 - 0 of 0

SQL ▼ 1 / 1 1 - 2 of 2

student_name	course_name
Alice	Database Systems
Bob	Database Systems

SQL ▼ 1 / 1 1 - 0 of 0

SQL ▼ 1 / 1 1 - 1 of 1

course_name	total_registrations
Database Systems	2